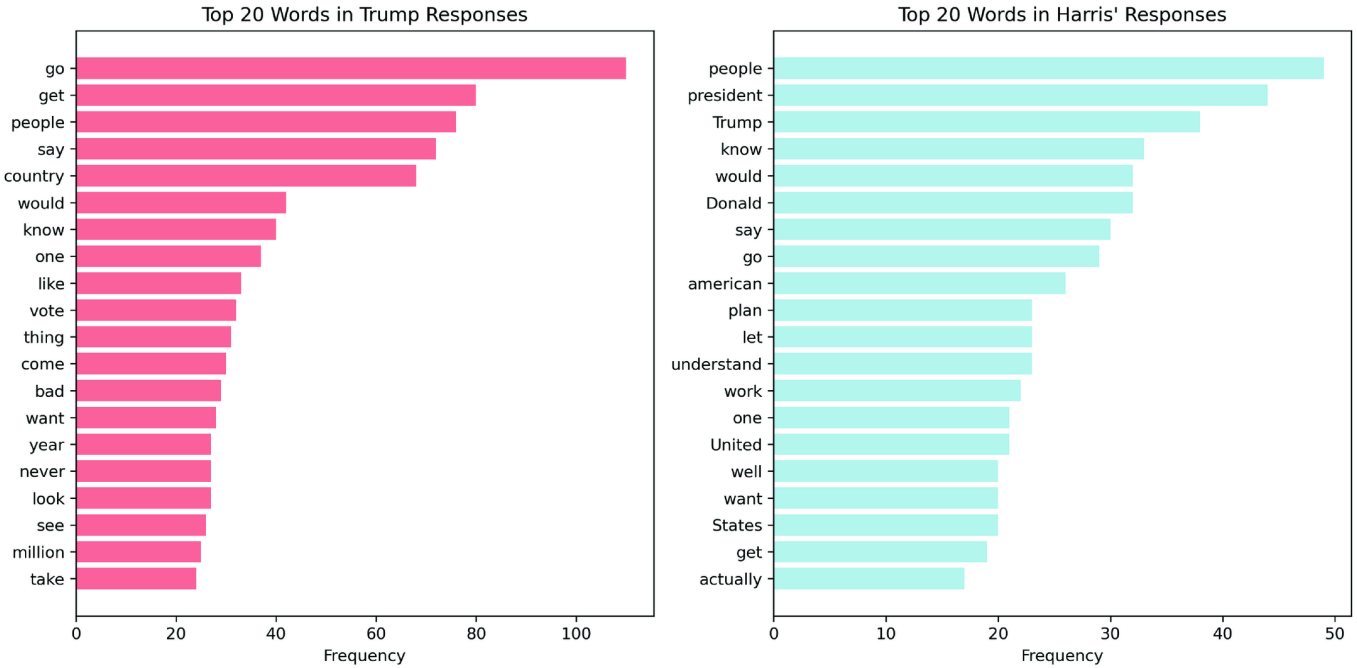
Lecture 9
Textual Data: Vector Space Model and TF-IDF

Dennis Sun
Stanford University

DATASCI 112

January 26, 2026



Red and blue language: Word choices in the Trump and Harris 2024 presidential debate

1 Review

2 Vector Space Model

3 tf-idf

1 Review

2 Vector Space Model

3 tf-idf

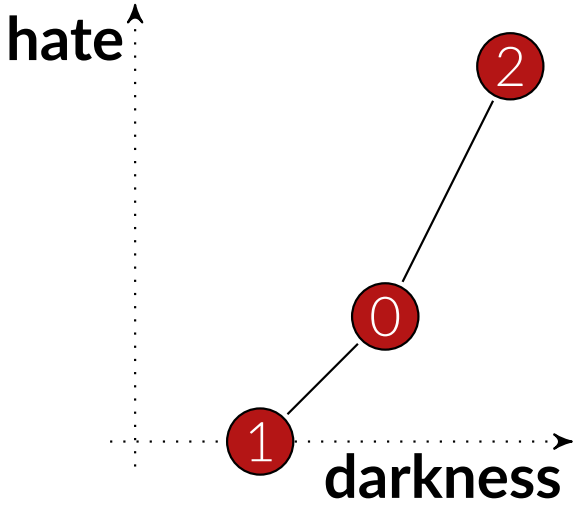
Textual Data

0. "Whoever has hate for his brother is in the darkness and walks in the darkness." —1 John 2:11
1. "Hello darkness, my old friend." —Simon & Garfunkel
2. "Returning hate for hate multiplies hate, adding deeper darkness to a night already devoid of stars. Darkness cannot drive out darkness; only light can do that." —MLK

$\Rightarrow$

	darkness	**hate**	…
0	2	1	…
1	1	0	…
2	3	3	…

Which document is most similar to document 0?



Using Euclidean distance, document 1 appears closer than document 2!

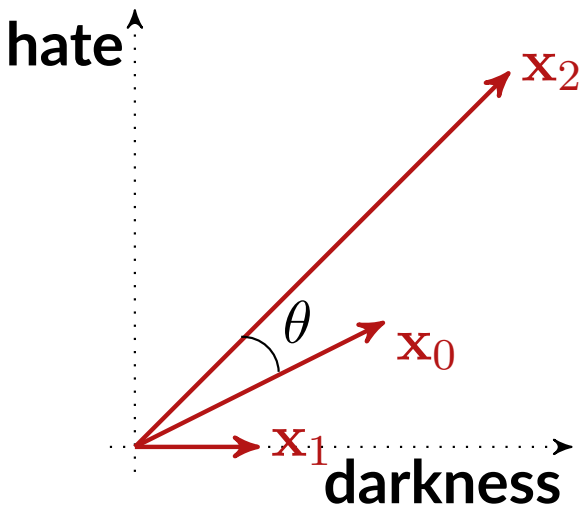
1 Review

2 Vector Space Model

3 tf-idf

Vector Space Model

In the **vector space model**, documents are represented as *vectors* instead of points.



The **length of a vector** is its distance from the origin $\mathbf{0}$:

$$||\mathbf{v}|| = \sqrt{\sum_{j=1}^{|V|} v_j^2}.$$

The distance between two vectors corresponds to the angle between them:

$$d(\mathbf{x}, \mathbf{x}') = 1 - \cos\theta = 1 - \frac{\sum_{j=1}^{|V|} x_j x'_j}{||\mathbf{x}||\ ||\mathbf{x}'||}.$$

Using cosine distance, document 2 now appears closer!

Implementing the Vector Space Model

```
documents = [
    "whoever has hate for his brother is in the darkness and walks in the darkn
    "hello darkness my old friend",
    "returning hate for hate multiplies hate adding deeper darkness to a night
]
```

First, we use Pandas to get the term-frequency matrix.

```
import pandas as pd
from collections import Counter

tf = pd.DataFrame(
    [Counter(doc.split()) for doc in documents],
).fillna(0)
tf
```

	whoever	has	hate	for	...	light	can	do	that
0	1.0	1.0	1.0	1.0	...	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0
2	0.0	0.0	3.0	1.0	...	1.0	1.0	1.0	1.0

3 rows × 35 columns

Now we just have to implement the formula for cosine distance.

Implementing the Vector Space Model

tf =

	whoever	has	hate	for	...	light	can	do	that
0	1.0	1.0	1.0	1.0	...	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0
2	0.0	0.0	3.0	1.0	...	1.0	1.0	1.0	1.0

3 rows × 35 columns

Now we just have to implement the formula for cosine distance.

$$d(\mathbf{x}, \mathbf{x}') = 1 - \frac{\sum_{j=1}^{|V|} x_j x'_j}{||\mathbf{x}||\ ||\mathbf{x}'||}.$$

```
import numpy as np

def length(v):
  return np.sqrt((v ** 2).sum())

def cos_dist(v, w):
  return 1 - (v * w).sum() / (length(v) * length(w))

cos_dist(tf.loc[0], tf.loc[1]), cos_dist(tf.loc[0], tf.loc[2])
```

```
(0.8048199854102933, 0.6460038372976056)
```

Vector Space Model in Scikit-Learn

It's always easier to do it in Scikit-Learn.

```
from sklearn.feature_extraction.text import CountVectorizer

vec = CountVectorizer(token_pattern=r"\w+")
vec.fit(documents)
tf_matrix = vec.transform(documents)
tf_matrix.todense()
```

```
matrix([[0, 0, 0, 1, 1, 0, 0, 2, 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 2, 1, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 1, 1],
        [0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0,
         0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [1, 1, 1, 0, 0, 1, 1, 3, 1, 1, 1, 1, 1, 0, 0, 3, 0, 0, 0, 0, 1,
         1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0]])
```

```
from sklearn.metrics import pairwise_distances
pairwise_distances(tf_matrix[0, :], tf_matrix,
                   metric="cosine")
```

```
array([[0.        , 0.80481999, 0.64600384]])
```

1 Review

2 Vector Space Model

3 tf-idf

tf-idf

So far, we've simply counted the **term frequency** $\mathrm{tf}(d, t)$: how many times each term t appears in each document d.

<u>Problem:</u> Common words like "is" or "the" tend to dominate because they have high counts.

We need to adjust for how common each word is:

1. Count the fraction of documents the term appears in:

$$\mathrm{df}(t, D) = \frac{\#\text{ documents containing term } t}{\#\text{ documents}} = \frac{|\{d \in D : t \in d\}|}{|D|}$$

2. Invert and take a log to obtain **inverse document frequency**:

$$\mathrm{idf}(t, D) = 1 + \log \frac{1}{\mathrm{df}(t, D)}.$$

3. Multiply tf by idf to get tf-idf:

$$\text{tf-idf}(d, t, D) = \mathrm{tf}(d, t) \cdot \mathrm{idf}(t, D).$$

Now we can use the **tf-idf matrix** just like we used the term-frequency matrix.

tf-idf by Hand

The term-frequency matrix for this corpus is:

0. "Whoever has hate for his brother is in the darkness and walks in the darkness."
1. "Hello darkness, my old friend."
2. "Returning hate for hate multiplies hate, adding deeper darkness to a night already devoid of stars. Darkness cannot drive out darkness; only light can do that."

$\Rightarrow$

	darkness	**hate**	...
0	2	1	...
1	1	0	...
2	3	3	...

Now let's calculate the tf-idf matrix!

1. Calculate the document frequencies:

$$\text{df}(\text{“darkness”}, D) = \frac{3}{3} = 1 \qquad \text{df}(\text{“hate”}, D) = \frac{2}{3}$$

2. Calculate the inverse document frequencies:

$$\text{idf}(\text{“darkness”}, D) = 1 + \log 1 = 1 \quad \text{idf}(\text{“hate”}, D) = 1 + \log \frac{3}{2} \approx 1.405$$

3. Multiply tf by idf to get tf-idf:

	darkness	**hate**	...
0	2	1.405	...
1	1	0	...
2	3	4.216	...

tf-idf in Scikit-Learn

```
from sklearn.feature_extraction.text import TfidfVectorizer

# The options ensure that the numbers match our example above.
vec = TfidfVectorizer(smooth_idf=False, norm=None)
vec.fit(documents)
tfidf_matrix = vec.transform(documents)
```

Now we can use this tf-idf matrix just as we used the term frequency matrix!

```
pairwise_distances(tfidf_matrix[0, :], tfidf_matrix,
                   metric="cosine")
```

```
array([[0.        , 0.94612045, 0.84453506]])
```

Dr. Seuss Example

Let's go into Colab and find the Dr. Seuss book that is most similar to *One Fish*, *Two Fish*, *Red Fish*, *Blue Fish*.